1 Research paper deep dive

1.1 Interpretation of the Model Described in the Paper

1.1.1 Assumptions Made

For the purposes of this assignment, it is worth clarifying my interpretation of the structure of the model as described in the paper [1].

In the paper, Weng introduces the fact that the following components are included in their model:

- Cubic spline interpolation where datapoints are missing
- Downsampling of the data to 1 Hz
- Extraction of three features (E_{sw}, R_{th}, V_{th})
- A dilated CNN
- An LSTM
- A cross-attention mechanism
- A "physical constraint" loss function of $\mathcal{L} = 0.7 * MSE(RUL_{pred}, RUL_{true}) + 0.3 * |RUL_{pred} - N_f|$

Nearly everything else about the model is left to the interpretation of the reader. In the completion of this project, the following interpretation has been made:

- Cubic spline interpolation where datapoints are missing
- Downsampling of the data to 1 Hz
- Extraction of three features (E_{sw}, R_{th}, V_{th})
- A dilated CNN, taking in the time series values from E_{sw}.
- An LSTM, taking in the R_{th} and V_{th}
- A cross-attention mechanism taking in the output of both the CNN (as the query) and LSTM (as keys and values)
- A "physical constraint" loss function of $\mathcal{L} = 0.7 * MSE(RUL_{pred}, RUL_{true}) + 0.3 * mean(|RUL_{pred} - N_f|)$. This is not used except in the comparison of loss functions, as just the MSE seemed to work better.
- A single fully-connected layer from the output of the cross attention layer to a single scalar "RUL" output.

The CNN and attention mechanisms were implemented to be causal.

For the purposes of this assignment, no synthetic data was generated. The dataset only includes data for the individual tests of 4 devices (labeled 2-5). Device 2 is chosen as the "dev set" and the other devices' data is used in training.

Lastly, because the RUL values, given in seconds, are extremely large, the model worked best when scaling these down by a factor of 2000.

1.1.2 Code

The data processing code was common between all tests described in this assignment. Data processing included cubic spline interpolation, downsampling, and extraction of the three features described in the paper.

Each test is self-contained in its own ipynb file.

All code is in a GitHub repo, linked here: https://github.com/snorklerjoe/lifetime-prediction-machine-learning This repo also contains the fully preprocessed data as well.

[1] https://doi.org/10.54254/2755-2721/2025.22570

1.2 Network Design: Multiple Dilation Factors

The paper shows a significant improvement in training and validation loss when using a dilated CNN over a CNN. In analyzing the R_{th} data used by the CNN, there appear to be multiple frequency components. Because of this, I propose an approach involving multiple CNNs with different amounts of dilation in parallel, then combined through a fully-connected linear layer.

To investigate this, two models are created. To avoid confusing gains from the fully-connected layer as improvements due to the varied dilation, the control model (without varied dilation) contains the same three CNNs in parallel and combined through a fully-connected layer with the same hyperparameters other than the amount of dilation. One could expect this to better capture information relating to different frequency components within the data, however it is possible that the added complexity and additional parameters might make the model more susceptible to issues with overfitting.

For the comparison, both models involve three CNNs in parallel (taking the same input), with a fully-connected linear layer bringing the three concatenated outputs down from a total dimension of $3 * \textit{hidden size}$ to the hidden size. Both models still involve three CNNs and a fully-connected layer such that the experimental results are not influenced by the large number of added parameters and fully-connected layer. The control model uses the same dilation factor, 256 for each of the CNNs. The alternative model uses the dilation factors $32, 63, 256$ for the three CNNs.

All code for this experiment is contained in varied_dilation.ipynb.

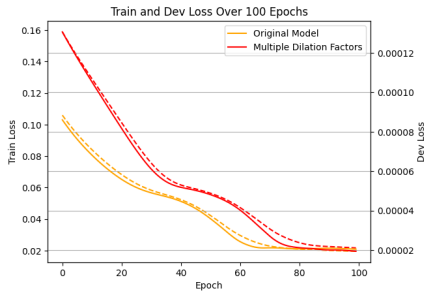

Figure 1: Training and dev loss for both model types, with train loss shown as a dashed line.

Figure 2: Evaluation of the two models over the data in the dev set.

The experimental results as shown in Figure 1 demonstrate that it takes longer to train the model with multiple dilation factors, which may be due to the increased heavy-lifting of the fully-connected layer after the CNNs, as it needs to combine information from different frequency components. Once trained, the two models perform similarly in terms of loss function values— The modified model shows an increase of training loss from 0.0196 to 0.0217, and a decrease in dev loss from 0.0225 to 0.0214. The differences are subtle, but the reduced dev loss suggests that the modified model may be less prone to overfitting. It is also possible that this is just due to the original model converging faster and overfitting due to the additional training time until the modified model converges.

However, when evaluating the model on the dev set, as shown in Figure 2, there is an improvement for much of the data. The predicted RUL with the model with multiple dilation factors appears less erratic, particularly visible in the spike in RUL at around 400 seconds. The data in this dataset are far from perfect— the read-me's that accompany it mention instrumentation failures and sensor drift. As a result, they are representative of any real-world data. It appears that the model with multiple dilation factors is able to pick up on elements of the data that are more correlated with actual changes in the RUL as opposed to noise and other issues.

In the paper, Weng shows an improvement with using a dilated CNN over one without dilation. The paper argues that simply using a dilated CNN allows the model to pick up on "features of varying scales." It appears that using multiple CNNs with different amounts of dilation only enhances this ability further.

Because the experimental data suggest the modified model may be less susceptible to overfitting and capable of picking up on trends of various frequencies within the data, this approach is superior. Although it did take slightly longer to train (as shown in Figure 1), all of this occurs with very small data sets (training on just three devices worth of data, when much simpler curve-fitting predictions in industry tend to be made with data from several dozen devices), and yet the model shows quite acceptable performance given this constraint.

1.3 Transfer Learning to More Directly/Immediately Model Device Self-Heating

The data involved in reliability tests typically include long time-series sensor and instrumentation data. This paper uses a few physics-derived features that are intuitively relevant to reliability. The paper uses E_{sw}, essentially a power dissipation measure, as the input to the CNN. In semiconductor reliability, temperature (which has to do with thermal resistance and power dissipation) is extremely correlated with the acceleration of damage to a device.

In the same vein as the physics / intuition-derived feature extraction, I propose attempting to model the device temperature using a CNN, and using that pretrained CNN as the CNN portion of the paper's model. This application of transfer learning may help the model to more easily learn patterns more directly associated physically with device wear-out.

This scheme should also allow the model to utilize thermal resistance information in a more useful way. The R_{th} feature taken in by the LSTM portion of the model is derived from steady-state measurements rather than those related directly to the self-heating of the device during transient pulses that occur throughout the test. Because the R_{th} that is typically relevant to semiconductor reliability measurements has everything to do with self-heating, modeling the temperature of the device with respect to self-heating may be far more useful to modeling the device's RUL.

In this experiment, a CNN of the same architecture as used in the paper's model was used, with a single fully-connected layer to a scalar output, to model temperature. This model was trained using the E_{sw} feature as input, and the *time_domain_packageTemperature* column from the data to train the output of the model. The *time_domain_internalTemperature* data column did not appear to be useful, perhaps due to measurement error or improper scaling. If *time_domain_internalTemperature* was also measured accurately in degrees C, it would be a better measurement to use for training this network, as the goal is to have the model learn information about the self heating of the device itself.

After training, the fully-connected layer was removed, and a separate *hidden_size* x *hidden_size* fully connected layer was added to the end of the pretrained model, and this was inserted as the CNN portion of the model described in the paper. The full RUL model was then trained. The same model architecture was used for two separate models. The control model did not receive the weights from the temperature-modeling CNN, and used randomly initialized weights for the CNN instead.

All code for this experiment is contained in temp_xfer_learning.ipynb.

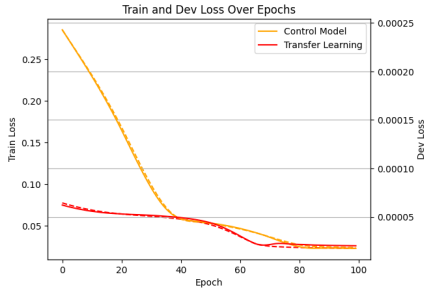

Figure 3: Training and dev loss for both model types, with train loss shown as a dashed line.

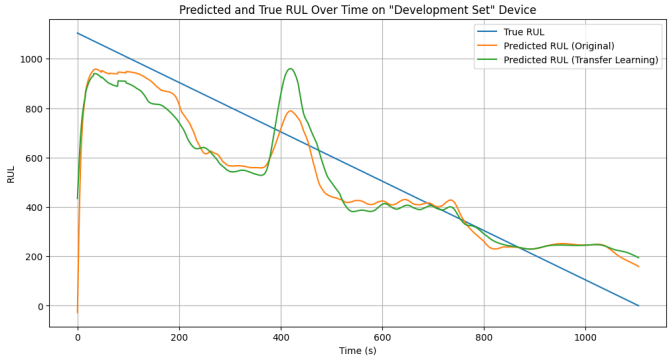

Figure 4: Evaluation of the two models over the data in the dev set.

The data from the experiment show that this transfer learning method does not cause the model learning to stumble upon a vastly different minimum loss. Figure 3 shows that the model with the pre-trained weights for the CNN started with a somewhat lower training and dev loss, but that may also likely be due to different random initialization of weights in the LSTM or attention mechanism.

In predicting the RUL of the dev device, as shown in Figure 4, it appears that the model without the transfer learning technique is actually superior in being closer to modeling the actual RUL.

Although self-heating is inherently tied with the information from the transient waveform data in the dataset, it appears from this experiment that the simplicity of the model suggested in the paper is more appropriate.

With more features than just E_{sw} and a better measure of the device's internal temperature, this method might work better. However, with the data provided in the dataset used in this paper, Weng's method is both simpler and better-performing.

1.4 Loss Function

This paper utilizes a weighted average of 70% mean-squared error with respect to the training data with 30% of a loosely predicted remaining useful life with a well-known approximate model, the Coffin-Manson equation. This equation, $N_f = A(\Delta T)^\alpha$, is often used to model reliability of devices in some scenarios including that under which this dataset was made. This is likely beneficial due to the amount of noise in the (relatively small amount of) data used in this paper. In addition, the model used in the loss function is one that would be appropriate for a simpler though less accurate way of modeling the RUL of the device without deep learning.

Notably, the model used in this case is chosen with some arbitrary constants ($A = 0.05$, $\alpha = -2.5$) to fit it, roughly, to the situation and materials used. It would be interesting to try utilizing this model in the loss function with a specific curve fit to the data, so as to more reasonably utilize this model– for each training step, the model can be fit and the loss function can be chosen for the specific piece of data being used. This may allow for capturing the overall intuition baked into the Coffin-Manson equation while also capturing other nuances with greater agility.

For this experiment, the loss function was modified to use mean-squared error instead of mean-absolute error for the loss with respect to the Coffin-Manson equation for consistency:

$\mathcal{L} = 0.7 * MSE(RUL_{pred}, RUL_{true}) + 0.3 * MSE(RUL_{pred}, A(\Delta T)^\alpha)$

The experiment was constructed by using a training function that incorporates the Coffin-Manson equation with, for the experimental model, a pair of 1-by-1 linear layers for scaling A and α to the data during training.

For consistency and to avoid differences in the data as a result of merely random initialization of weights, all of the weights in one model were randomly initialized, and the second model was cloned with the same initial random weights. Both models were then trained, one with the loss function with the arbitrary $A = 0.1$ (modified to fit temporal scaling of the data) and $\alpha = -2.5$, and the other with the adaptively fit A and α.

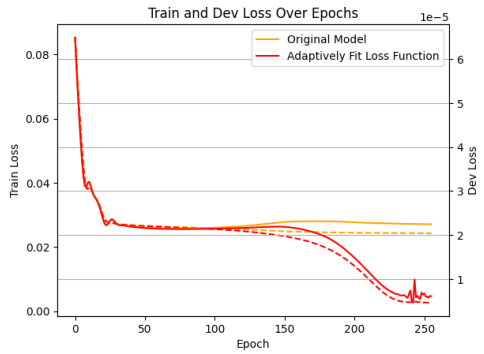

Figure 5: Training and dev loss for both model types, with train loss shown as a dashed line.

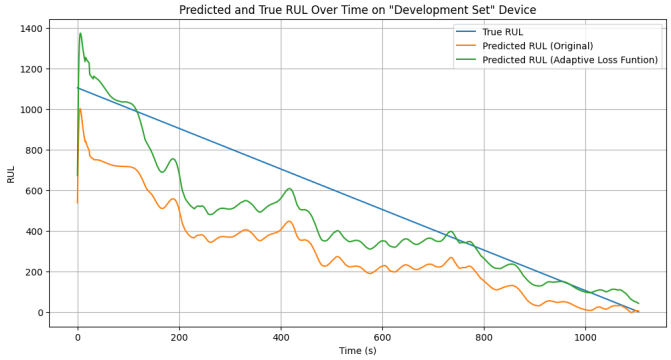

Figure 6: Evaluation of the two models over the data in the dev set.

Figure 5 shows that the adaptively fit Coffin-Manson parameters allow the model to perform much better. A bit of a ripple is seen in the training and dev losses towards the end of training, possibly due to some feedback mechanism from the loss function being adaptively modified during training.

Figure 6 shows that the overall RUL predictions from the model trained with the adaptively fit loss function is shifted much closer to the actual true RUL values. This suggests that the arbitrary constants chosen in the paper, while allowing the model to learn the approximate behavior of the Coffin-Manson equation, also imparts error due to the arbitrary parameters not truly fitting the experimental results seen in the data. Utilizing the adaptively-fit loss function allows the model to still generally learn information from the Coffin-Manson equation while not misrepresenting the actual deviation from the data being modeled.

The paper suggested using the Coffin-Manson equation in the loss function for the purpose of helping the model learn the patterns in the data associated with traditional means of understanding semiconductor reliability. This modified means of training using the adaptive model performs much better and better allows the model to learn the generalization of its behavior rather than any specific values that did not come from the data.

1.5 Disclosure of LLM Usage

Gemini, Claude, and Github Copilot were used in the writing of code for this assignment. The data preprocessing pipeline was carefully described to GitHub Copilot to generate the code for the data preprocessing script, *data.py*,

which took the data from Matlab data files into a more nicely-formatted csv, performed the interpolation and downsampling, as well as feature extraction.

Claude was extremely helpful in debugging the model when replicating the paper's model. It suggested scaling features and output, as well as suggested I train sections of time series data all at once with a causal model rather than train the model with sliding windows. It also came in handy for assisting with optimizations to make training occur faster, such as relying less upon slower pandas operations with my data in dataframes and more upon torch and numpy operations.